Why late timesteps dominate the XOPD per-step loss

A flow-matching account of the `d_k` blow-up, artifact vs. signal, and the `xopd_dk_space` reweighting

Flow Factory

Abstract

Theory companion to the empirical observation `docs/xopd/progress/2026-07-12-per-timestep-dk-dominanc` (last quarter of steps ≈ 81% of $\sum_k d_k$, final step ≈ 40%); the implemented remedy is `train.xopd_dk_space="x0"` (clean-latent loss, §5). Relevant code: `compute_per_step_kl` in `src/flow_factory/trainers/xopd/common.py`, called by `_precompute_d_per_timestep` / `_optimize_train_pass` in `src/flow_factory/trainers/xopd/trainer.py`.

This note defines three per-step MSE losses (L_{xt}, L_v, L_{x_0}), their exact conversion on the FLUX.2-klein 28-step grid, and why the default L_{xt} (config `"xt"`) dominates late timesteps. Companion data: §3–§4.

Contents

1 Three per-step MSE losses and their exact conversion

XOPD compares student and teacher on the *same* on-policy rollout state x_t at step index k; only their predictions differ. Under rectified-flow interpolation ($x_t = (1-\sigma)z_0 + \sigma\varepsilon$, $\sigma = t/1000$) and ODE Euler stepping ($\mu = x_t + v\,\Delta t$, $\Delta t = \sigma_{k+1}^{\text{sched}} - \sigma_k^{\text{sched}} < 0$), there are three equivalent MSE targets (config `xopd_dk_space`, default `"xt"`):

knob	loss	formula	note
`xt`	L_{xt}	$\|\mu_s - \mu_t\|^2$	transition mean / next latent (default)
`v`	L_v	$\|v_s - v_t\|^2$	raw velocity MSE; ODE-only
`x0`	L_{x_0}	$\|z_{0,s} - z_{0,t}\|^2$	clean-latent MSE; ODE-only

Ratio. $\mathrm{MSE}(v) : \mathrm{MSE}(xt) : \mathrm{MSE}(x0) = 1 : \Delta t^2 : \sigma^2$.

Here μ is the one-step transition mean ($x_{t+\Delta t}$ under ODE), v the model velocity (`noise_pred`), and $z_0 = x_t - \sigma v$ the analytically recovered clean latent. Because x_t cancels:

$$\mu_s - \mu_t = \Delta t\,(v_s - v_t), \tag{1}$$

$$z_{0,s} - z_{0,t} = -\sigma\,(v_s - v_t), \tag{2}$$

$$\Rightarrow \quad L_{xt} = \Delta t^2\, L_v, \tag{3}$$

$$L_{x_0} = \sigma^2\, L_v, \tag{4}$$

$$L_{x_0} = \left(\tfrac{\sigma}{\Delta t}\right)^2 L_{xt}, \tag{5}$$

$$L_v = \left(\tfrac{\Delta t}{\sigma}\right)^2 L_{x_0}. \tag{6}$$

All four identities are *exact* on the discrete ODE grid (no approximation). The conversion factors depend *only* on $(\sigma_k, \Delta t_k)$ at step k; they do not require equal errors across steps — they convert *the same* underlying $(v_s - v_t)$ disagreement between spaces.

Conversion matrix (multiply to go from column loss to row loss).

	$\to L_{xt}$	$\to L_v$	$\to L_{x_0}$
from L_{xt}	1	Δt^{-2}	$(\sigma/\Delta t)^2$
from L_v	Δt^2	1	σ^2
from L_{x_0}	$(\Delta t/\sigma)^2$	σ^{-2}	1

What each loss implicitly weights (if you sum $\sum_k L_{\{\cdot\},k}$ with coefficient 1). Fix the *underlying* velocity disagreement $L_{v,k}$ at step k:

- L_{xt} **(default):** step k contributes $\Delta t_k^2 L_{v,k}$ — late steps have larger $|\Delta t_k|$ and dominate (§3, col. "$\%L_{xt}$").
- L_v **(`xopd_dk_space="v"`):** step k contributes $L_{v,k}$ directly — the native FM velocity target; under the fixed 28-step grid the per-step Δt_k vary only modestly so this is close to uniform (§4).
- L_{x_0} **(`xopd_dk_space="x0"`):** step k contributes $\sigma_k^2 L_{v,k}$ — early high-noise steps ($\sigma \approx 1$) dominate (col. "pct L_{x_0}").

Alternatively, fix $L_{x_0,k}$ (equal clean-latent disagreement): $L_{xt,k} = (\Delta t_k/\sigma_k)^2 L_{x_0,k}$, which is the "$(\Delta t/\sigma)^2$ blow-up" narrative (§6).

2 The key identity (legacy shorthand)

With `normalize_d_k=false`, the logged `train/d_k/{ti}` is L_{xt} (not L_v). Combining (3)–(6):

$$L_{xt} = \Delta t^2 L_v = \left(\tfrac{\Delta t}{\sigma}\right)^2 L_{x_0}.$$

So the default loss is clean-latent disagreement multiplied by the implicit per-step weight $(\Delta t/\sigma)^2$ (§1). The x-space knob sets L_{x_0} directly and removes that factor (§5).

3 Concrete schedule: official FLUX.2-klein @ 28 steps

We reproduce the *exact* denoising grid used in XOPD training (`num_inference_steps=28`, 512×512) by loading the default scheduler from the official HuggingFace repo and calling the same `retrieve_timesteps` path as `Flux2KleinPipeline` (shifted flow-matching sigmas with `image_seq_len=1024`, $\mu = 1.859$):

```
FlowMatchEulerDiscreteScheduler.from_pretrained(
    "black-forest-labs/FLUX.2-klein-base-4B", subfolder="scheduler")
# sigmas = linspace(1, 1/28, 28); mu = compute_empirical_mu(1024, 28)
```

Repro script: `.scratch/flux2_klein_scheduler_timesteps.py` (uv venv `/Users/bowenping/.venvs/flux2-` `diffusers==0.39.0`).

Notation at step index `ti`. `ti=0` is the noisiest state; `ti=27` is the last denoising step before $\sigma = 0$. The scheduler returns a *timestep* $t_k \in [0, 1000]$ and a *shifted flow sigma* $\sigma_k^{\text{sched}} \in [0, 1]$. XOPD's clean-latent recovery uses the repo's FM fraction $\sigma_k = t_k/1000$ (`_noise_fraction`, `flow_match_sigma`). The ODE step size is $\Delta t_k = \sigma_{k+1}^{\text{sched}} - \sigma_k^{\text{sched}} < 0$ (from `scheduler.step`). On this grid $\sigma_k \approx \sigma_k^{\text{sched}}$ to machine precision.

Per-step conversion factors on the 28-step grid. Table 1 lists representative steps; Table 2 (auto-generated, `docs/xopd/flux2_klein_28step_loss_scales.tex`) gives all 28 rows. Repro: `.scratch/flux2_klein_scheduler_timesteps.py`.

`ti`	t	σ	Δt	$\frac{L_{xt}}{L_v} = \Delta t^2$	$\frac{L_{x_0}}{L_v} = \sigma^2$	$\frac{L_{x_0}}{L_{xt}}$	pct L_{xt}	pct L_{x_0}
0	1000.0	1.0000	−0.00574	3.3×10^{-5}	1.000	3.0×10^{4}	0.04%	5.28%
7	950.6	0.9506	−0.00929	8.6×10^{-5}	0.904	1.0×10^{4}	0.10%	4.77%
13	881.0	0.8810	−0.01584	2.5×10^{-4}	0.776	3.1×10^{3}	0.28%	4.10%
21	681.5	0.6815	−0.04505	2.0×10^{-3}	0.464	2.3×10^{2}	2.27%	2.45%
24	516.8	0.5168	−0.08175	6.7×10^{-3}	0.267	40.0	7.47%	1.41%
25	435.1	0.4351	−0.10456	1.1×10^{-2}	0.189	17.3	12.21%	1.00%
26	330.5	0.3305	−0.13847	1.9×10^{-2}	0.109	5.70	21.42%	0.58%
27	192.1	0.1921	−0.19206	3.7×10^{-2}	0.037	1.00	41.21%	0.19%

Table 1: FLUX.2-klein-base-4B, `num_inference_steps=28`, 512px. Columns 5–7: exact factors from (3)–(5). "$\%L_{xt}$" ("$\%L_{x_0}$"): share of $\sum_k L_{xt,k}$ (resp. $\sum_k L_{x_0,k}$) if $L_{v,k} \equiv$ const across steps.

Reading the numbers.

- **At `ti=27`, all three losses coincide in scale.** $|\Delta t_{27}| \approx \sigma_{27}$, so $\Delta t_{27}^2 = \sigma_{27}^2 \approx 0.037$ and $L_{xt} = L_{x_0} = L_v$ for the *same* velocity gap.
- **Default L_{xt} dominance ≈ flat L_v.** If $\|v_s - v_t\|^2$ were identical at every step, $\sum_k L_{xt,k}$ puts **41**% on `ti=27` (col. "$\%L_{xt}$"). The mixed XOPD run observes **40**% on `ti=27` — the logged curve matches *roughly constant velocity error* amplified only by Δt_k^2, not the stronger $(\Delta t_k/\sigma_k)^2$ factor (which would give **77**% if $L_{x_0,k}$ were constant).
- **L_{x_0} up-weights early steps.** With flat L_v, high-noise steps each take $\sim 5\%$ of $\sum L_{x_0}$ while `ti=27` takes only **0.19**%. Setting `xopd_dk_space="x0"` switches to this allocation.
- **Cross-step ratios (same L_v).** $L_{xt,27}/L_{xt,13} = (\Delta t_{27}/\Delta t_{13})^2 \approx$ **148**×; $L_{x_0,0}/L_{x_0,27} = \sigma_0^2/\sigma_{27}^2 \approx$ **27**×.
- **"Late step" $\neq \sigma \to 0$.** The last rollout step has $t = 192$ ($\sigma = 0.19$), not $t \approx 0$. Dominance comes from the discrete grid's large $|\Delta t_k|$ at the end.

ti	t	σ	Δt	$\frac{L_{xt}}{L_v}$	$\frac{L_{x0}}{L_v}$	$\frac{L_{x0}}{L_{xt}}$	pct L_{xt}	pct L_{x0}
0	1000.0	1.0000	-0.00574	3.3e-05	1.000	30380.38	0.04%	5.28%
1	994.3	0.9943	-0.00611	3.7e-05	9.89e-01	26518.80	0.04%	5.22%
2	988.2	0.9882	-0.00651	4.2e-05	9.76e-01	23039.00	0.05%	5.15%
3	981.6	0.9816	-0.00696	4.8e-05	9.64e-01	19912.21	0.05%	5.09%
4	974.7	0.9747	-0.00745	5.6e-05	9.50e-01	17114.06	0.06%	5.01%
5	967.2	0.9672	-0.00800	6.4e-05	9.36e-01	14621.44	0.07%	4.94%
6	959.2	0.9592	-0.00861	7.4e-05	9.20e-01	12410.80	0.08%	4.86%
7	950.6	0.9506	-0.00929	8.6e-05	9.04e-01	10460.53	0.10%	4.77%
8	941.3	0.9413	-0.01006	1.0e-04	8.86e-01	8748.98	0.11%	4.68%
9	931.3	0.9313	-0.01093	1.2e-04	8.67e-01	7256.11	0.13%	4.58%
10	920.3	0.9203	-0.01192	1.4e-04	8.47e-01	5962.25	0.16%	4.47%
11	908.4	0.9084	-0.01305	1.7e-04	8.25e-01	4849.23	0.19%	4.35%
12	895.4	0.8954	-0.01434	2.1e-04	8.02e-01	3899.25	0.23%	4.23%
13	881.0	0.8810	-0.01584	2.5e-04	7.76e-01	3095.63	0.28%	4.10%
14	865.2	0.8652	-0.01758	3.1e-04	7.49e-01	2422.59	0.35%	3.95%
15	847.6	0.8476	-0.01963	3.9e-04	7.18e-01	1865.26	0.43%	3.79%
16	828.0	0.8280	-0.02205	4.9e-04	6.86e-01	1409.55	0.54%	3.62%
17	805.9	0.8059	-0.02496	6.2e-04	6.50e-01	1042.44	0.70%	3.43%
18	781.0	0.7810	-0.02849	8.1e-04	6.10e-01	751.66	0.91%	3.22%
19	752.5	0.7525	-0.03281	1.08e-03	5.66e-01	525.93	1.20%	2.99%
20	719.7	0.7197	-0.03821	1.46e-03	5.18e-01	354.83	1.63%	2.73%
21	681.5	0.6815	-0.04505	2.03e-03	4.64e-01	228.85	2.27%	2.45%
22	636.4	0.6364	-0.05391	2.91e-03	4.05e-01	139.37	3.25%	2.14%
23	582.5	0.5825	-0.06567	4.31e-03	3.39e-01	78.683	4.82%	1.79%
24	516.8	0.5168	-0.08175	6.68e-03	2.67e-01	39.970	7.47%	1.41%
25	435.1	0.4351	-0.10456	1.09e-02	1.89e-01	17.315	12.21%	1.00%
26	330.5	0.3305	-0.13847	1.92e-02	1.09e-01	5.698	21.42%	0.58%
27	192.1	0.1921	-0.19206	3.69e-02	3.69e-02	1.000	41.21%	0.19%

Table 2: Three per-step MSE losses on the FLUX.2-klein 28-step grid (512 × 512). Columns 5–7 are exact conversion factors at step k (on-policy, shared x_t). Last two columns: normalized share of $\sum_k L_{xt}$ (resp. $\sum_k L_{x_0}$) if $\|v_s - v_t\|^2$ were identical at every step. Generated from `black-forest-labs/FLUX.2-klein-base-4B` default scheduler.

4 Empirical distribution: mixed run, converted from measured L_{xt}

Wandb run `4cnkluwk` (mixed geneval+OCR, `xopd_dk_space=xt`, `normalize_d_k=false`) logs $L_{xt,k}$ directly (§6). Under ODE we convert each step to the *hypothetical* L_v and L_{x_0} that would have been logged with the same underlying velocity gap:

$$L_{v,k} = \frac{L_{xt,k}}{\Delta t_k^2}, \qquad L_{x_0,k} = \left(\frac{\sigma_k}{\Delta t_k}\right)^2 L_{xt,k}.$$

Table 3 uses `mean_tail` from `progress/2026-07-12-per-timestep-dk-dominance.md` and the official 28-step $(\sigma_k, \Delta t_k)$ from §3.

aggregate	L_{xt} **(meas.)**	L_v **(conv.)**	L_{x_0} **(conv.)**
top-1 (`ti=27`)	40.2%	0.5%	0.02%
top-7 (`ti=21--27`, last 1/4)	81.0%	3.1%	0.9%
high-noise half (`ti=0--13`)	12.2%	91.5%	95.2%
low-noise half (`ti=14--27`)	87.8%	8.5%	4.8%

Table 3: Share of $\sum_k L_{\{\cdot\},k}$ (`mean_tail`) in each loss space. L_{xt} is measured; L_v and L_{x_0} are ODE-exact conversions using the FLUX.2-klein 28-step grid.

Reading.

- **Late dominance is specific to L_{xt}.** The observed 40%/81% concentration on late steps *disappears* under conversion to L_v (early half **92**%) or L_{x_0} (**95**%): the logged curve is dominated by Δt_k^2, while the underlying velocity gap is larger at high noise.
- L_v **is nearly flat in absolute scale** after conversion ($L_{v,0} \approx 1.26$ vs. $L_{v,27} \approx 0.056$, only **0.5**% of the sum on `ti=27`), consistent with Cause A (§6.1).
- **Ablation implication.** `xopd_dk_space="v"` removes the Δt_k^2 rescaling and should redistribute gradient toward high-noise steps; `"x0"` applies σ_k^2 and amplifies that further.

5 Implementation: the `xopd_dk_space` knob

Knob. `train.xopd_dk_space` $\in \{$`v`, `xt`, `x0`$\}$ (default `xt`). See `compute_per_step_kl` in `xopd/common.py`; `v` and `x0` require ODE (`XOPDTrainer.__init__` guard).

knob	per-step loss	relation to L_v
`xt`	$L_{xt} = \|\mu_s - \mu_t\|^2$	$L_{xt} = \Delta t^2 L_v$
`v`	$L_v = \|v_s - v_t\|^2$	identity
`x0`	$L_{x_0} = \|z_{0,s} - z_{0,t}\|^2$	$L_{x_0} = \sigma^2 L_v$

Since x_t cancels, $L_{x_0} = (\sigma/\Delta t)^2 L_{xt}$ ((5)). `normalize_d_k` applies only to `xt`; `v`/`x0` ignore it.

No extra cost. `x0` recovers z_0 analytically from μ; `v` uses $(\mu_s - \mu_t)/\Delta t$. **Logging.** `train/d_k/{ti}` values are in the selected space; compare *shapes* across spaces, not raw magnitudes (§4).

6 Two causes of the dominance: separate the artifact from the signal

6.1 Cause A — Δt_k^2 amplification in L_{xt} (a parameterization artifact)

The default loss is $L_{xt} = \Delta t^2 L_v$ ((3)), not velocity MSE. Summing $\sum_k L_{xt,k}$ with unit weight therefore applies a per-step factor Δt_k^2 to the underlying velocity gap. Late rollout steps have larger $|\Delta t_k|$ (Table 1), so L_{xt} **over-represents late steps even if L_v were flat**.

Quantitative check on the FLUX.2-klein 28-step grid (§3):

- If $L_{v,k} \equiv$ const: $\sum_k L_{xt,k}$ puts **41**% on `ti=27` (col. "%L_{xt}"). The mixed XOPD run observes **40**% on `ti=27` for logged L_{xt} — the curve matches *roughly constant velocity error* $\times \Delta t_k^2$, not a separate "mystery" effect.
- If instead $L_{x_0,k} \equiv$ const: $L_{xt,k} = (\Delta t_k/\sigma_k)^2 L_{x_0,k}$ would put **77**% on `ti=27`. The observed **50**$\times$ ratio $L_{\mu,27}/L_{\mu,13}$ is *between* the flat-L_v prediction ($\sim 148\times$ from Δt ratio alone, before L_v shrinkage) and the flat-L_{x_0} prediction ($\sim 3100\times$), because both the metric *and* the actual L_v vary with k.
- Switching to L_{x_0} (`xopd_dk_space="x0"`) replaces the Δt_k^2 weighting with σ_k^2 (col. "%L_{x_0}"), up-weighting high-noise structure steps.

This parallels diffusion training, where ε-prediction MSE needs SNR reweighting to avoid low-noise dominance; here the analogue of "native" FM regression is L_v, not L_{xt}.

6.2 Cause B — genuine difficulty near the data manifold (a real signal)

Near clean, the posterior $\mathbb{E}[z_0 \mid x_t]$ sharpens, the velocity field's spatial Jacobian (Lipschitz constant) grows $\sim 1/\sigma$, and the target encodes sharp, high-frequency, multimodal detail. Under `normalize_d_k=false` the latent scale is also largest at clean. This part of the late-step cost is **real** and tied to final-image quality.

6.3 The U-shape

The observation doc's minimum at ti$\approx$ 12–13 and mild high-noise bump at ti$=$ 0 follow from $L_{xt} = \Delta t^2 L_v$ with L_v largest at high noise: early steps have small Δt^2 but large L_v; late steps have large Δt^2 but shrinking L_v; the minimum is where these cross.

Corollary: `normalize_d_k=true` makes it worse, not better

`normalize_d_k=true` divides by $2\bar{\sigma}^2 = 2\,\mathrm{std\_dev}_t^2\,|\Delta t|$, and $\bar{\sigma} \to 0$ toward clean, stacking a *second* amplification on top of $1/\sigma^2$. This is why transition-KL objectives (DanceGRPO/DDPO-style) intrinsically favor the low-noise region. **To equalize timesteps you must go the opposite way.**

7 XOPD-specific reading (large teacher)

"XOPD benefits more because the teacher is larger" is correct, but the benefit lives in **Cause B**, not A:

- The 32B→4B teacher's advantage (fine detail, texture, aesthetics) concentrates in the *low-noise* steps, where on-policy x_t is already near the student's final sample — the most valuable place for a big teacher to "polish".
- Structural/semantic correctness (geneval count/position, OCR) is decided at *high-noise* steps.

So "large teacher" does **not** imply "train only late steps". It implies "the late-step Cause-B component is worth keeping — but do not mistake the Cause-A amplification for that value."

8 Unified view: every proposal is a choice of per-step weight w_k

All proposals choose how to weight the three losses (or an equivalent L_v) across steps: $\mathcal{L} = \sum_k w_k L_{\{xt,v,x_0\},k}$.

direction	loss / weight on L_v	effect
default	L_{xt}, $w_k = 1 \Rightarrow w_k L_v = \Delta t_k^2$	late-step dominance (41% on ti=27 if L_v flat)
1. late-only (DanceOPD)	L_{xt} on $\sigma_k < \tau$ only	polish; drops structure learning
2a. L_{x_0} (implemented)	L_{x_0}, $w_k = 1 \Rightarrow w_k L_v = \sigma_k^2$	up-weights high-noise steps
2b. L_v (implemented)	`xopd_dk_space="v"`	native FM velocity MSE
2c. EMA balance	$L_{xt}/\mathrm{EMA}(L_{xt})$	flattens logged scale; may suppress Cause B
3. endpoint-sensitivity	weight $\propto \partial z_0/\partial v_k$	2a is first-order proxy

8.1 Evaluation

- **Direction 1 (late-only)** trains L_{xt} on a late window only; same Δt_k^2 factor within the window. Drops high-noise steps where L_{x_0} (hence structure) matters most.
- **Direction 2a** (L_{x_0}) replaces Δt_k^2 with σ_k^2 on the shared L_v. **Direction 2b** (L_v) removes the Δt_k^2 factor entirely (§4). **Do not use `normalize_d_k=true`** to equalize (§6.3).
- **Direction 3 (sensitivity)** idealizes weighting L_v by endpoint Jacobian; L_{x_0} is a good first-order proxy.

Recommendation

Compare `xopd_dk_space` $\in$ {`xt`, `v`, `x0`} on OCR (§9): default `xt` vs. velocity `v` vs. clean-latent `x0`, plus late-only `xt` (direction 1).

9 Disambiguating experiment (artifact vs. signal)

The ablation in `docs/xopd/progress/2026-07-12-ablation-relay.md` (all on OCR, otherwise identical) is exactly the test:

variant	config	reads
full-timestep xt-MSE	`flux2_klein_32b_to_4b_l1_ocr_1kep.yaml`	default (`xt`)
late-timestep xt-MSE	`flux2_klein_32b_to_4b_l1_ocr_selective_teacher_1kep.yaml`	direction 1
full-timestep x0-MSE	`flux2_klein_32b_to_4b_l1_ocr_xspace_1kep.yaml`	direction 2a
full-timestep v-MSE	`flux2_klein_32b_to_4b_l1_ocr_vmse_1kep.yaml`	direction 2b

Interpretation:

- If `x0` or `v` **improves** high-noise-half learning and structure metrics *without* degrading clean-end detail → the original `xt` dominance was mostly **Cause A (artifact)**; reweighting is a net win.
- If `x0`/`v` **hurts** clean-end detail with insufficient structure gains → the late-step **Cause B (real value)** dominates; consider soft late-window `xt`.

Remark. *Logged `train/d_k/{ti}` magnitudes differ across `v`/`xt`/`x0`; compare per-timestep* shape *and downstream eval (§4), not raw values across spaces.*